Book A Doctor

Full Stack MERN Doctor Appointment Booking & Management System Documentation

1. Introduction

Project Title: Book A Doctor
Tech Stack: MongoDB, Express.js, React.js, Node.js (MERN)
Team Members: Sagar Kumar Shelake (Role: Full Stack Developer / Creator)

2. Project Overview

Purpose: Book A Doctor is a role-based healthcare web application designed to streamline the appointment booking process between patients and medical professionals. It bridges the gap by allowing users to register, apply for doctor profiles, get administrator approval, book time-slotted appointments, and receive real-time notifications.

Key Features: Role-based authentication (User, Doctor, Admin), dynamic doctor application & verification workflow, interactive calendar time-slot booking, appointment status tracking, and notification management.

3. Architecture

Frontend (React.js)

Single Page Application (SPA) leveraging React Router for navigation, Axios for RESTful API communication, component-based CSS styling, and reactive state hooks.

Backend & DB

Node.js & Express.js REST API server handling authentication middleware, Mongoose ODM schemas, and MongoDB Atlas cloud collections for persistence.

4. Setup Instructions & Prerequisites

Prerequisites: Node.js (v16+ recommended), npm, MongoDB Compass or MongoDB Atlas connection string.

Environment Variables: Create a `.env` file inside the root server directory containing `MONGO_URL` and `JWT_SECRET`.

5. Folder Structure

Client (Frontend)

`/src/components` - Layouts, Protected Routes
`/src/pages` - Login, Register, Home, Appointments, Apply Doctor, Admin Lists

Server (Backend)

`/models` - User, Doctor, and Appointment Schemas
`/routes` - Auth, User, Admin, and Doctor API routers
`server.js` - Entry point & Express server configuration

6. Running the Application

To run the application locally on your machine, execute the following commands in two separate terminal windows:

- **Backend Server:** Navigate to the root directory and run `npx nodemon server.js` (or `npm start`). Starts on port 5000.
- **Frontend Client:** Navigate to the `client` directory and run `npm start`. Starts on port 3000 (`http://localhost:3000`).

7. API Documentation (Core Endpoints)

Endpoint	Method	Description
`/api/user/register`	POST	Register a new user account (User/Admin)
`/api/user/login`	POST	Authenticate user and generate session token
`/api/user/apply-doctor-account`	POST	Submit doctor professional details for approval
`/api/admin/get-all-doctors`	GET	Admin retrieves list of pending/approved doctors
`/api/admin/change-doctor-status`	POST	Approve or block a registered doctor profile
`/api/user/book-appointment`	POST	Book a timed consultation slot with a doctor

8. Authentication & Authorization

The application implements secure token-based authentication using JSON Web Tokens (JWT) and encrypted passwords hashed via bcrypt. Protected route wrappers on the React frontend verify user roles before granting access to specific dashboards:

- **User / Patient Role:** Can browse approved doctors, check timing availability, and book or view appointments.
- **Doctor Role:** Access to manage personal practice schedules, view patient appointment requests, and accept/reject bookings.
- **Admin Role:** System-wide oversight to manage user accounts, review doctor credentials, approve applications, and monitor platform logs.

9. User Interface & Project Workflow

The UI is designed around clean navigational sidebars and card layouts. The core user flow involves three main operational pillars:

Application Workflow

1. Users register and log in.
2. Regular users submit doctor applications via the *Apply Doctor* form.
3. Admins review notifications and approve doctor statuses.

Booking & Management

1. Patients select a doctor, check calendar availability, and book slots.
2. Doctors receive appointment notifications.
3. Status updates reflect instantly in user dashboards.

10. Testing Strategy

Testing is conducted manually across all access tiers: verifying input field validations (e.g., checking required phone numbers and website syntax), testing database synchronization in MongoDB Compass during appointment creation, and verifying real-time notification states upon status updates.

11. Screenshots & Demo Reference

The application features responsive dashboard headers, dynamic modal notifications for unseen/seen alerts, interactive date-picker calendars, and status badges (pending, approved, blocked). Refer to the accompanying video demonstration for a complete walkthrough of user, doctor, and admin interactions.

12. Known Issues

- Time slot validations currently require strict adherence to doctor working hours; overlapping bookings require explicit manual confirmation.
- Date formatting in specific localized environments may occasionally show default date strings if timezone headers are unhandled.

13. Future Enhancements

- **Online Payment Integration:** Integrate gateways like Stripe or Razorpay for consultation fee prepayments during booking.
- **Video Consultations:** Embed WebRTC or Agora video calling directly into the patient/doctor appointment interface.
- **Automated Email Notifications:** Send appointment confirmation reminders via Nodemailer or SendGrid integration.
- **Medical History Records:** Allow doctors to attach digital prescriptions and patient medical notes post-consultation.